C04 -AT3 -DEBUGGING

Q1 **Inheritance and Constructor Order** 10 marks

```
#include <iostream>
using namespace std;

class Vehicle {
public:
Vehicle() {
cout << "Vehicle Constructor" << endl;
}
};

class Car : Vehicle {
public:
Car() {
cout << "Car Constructor" << endl;
}
};

int main() {
Car c;
return 0;
}
```

Your Code

```
#include <iostream>
using namespace std;
class Vehicle {
public:
    Vehicle() {
        cout << "Vehicle Constructor" << endl;
    }
};
class Car : public Vehicle {
public:
    Car() {
        cout << "Car Constructor" << endl;
    }
};
int main() {
    Car c;
    return 0;
}
```

Input

```
Vehicle
Car
```

Output

Q2 **Encapsulation and Setter Validation** 10 marks

```
#include <iostream>
using namespace std;

class BankAccount {
public:
double balance;

void deposit(double amount) {
balance += amount;
}

void withdraw(double amount) {
balance -= amount;
}

void display() {
cout << balance << endl;
}
};

int main() {
BankAccount account;
double initial, deposit, withdraw;
cin >> initial >> deposit >> withdraw;
```

Your Code

```
#include <iostream>
using namespace std;
class BankAccount {
public:
    double balance;
    void deposit(double amount) {
        balance += amount;
    }
    void withdraw(double amount) {
        balance -= amount;
    }
    void display() {
        cout << balance << endl;
    }
};
int main() {
    BankAccount account;
    double initial, deposit, withdraw;
    cin >> initial >> deposit >> withdraw;
    account.balance = initial;
    account.deposit(deposit);
    account.withdraw(withdraw);
    account.display();
    return 0;
}
```

Input

```
1000 500 200
```

Output

Q3 **Virtual Destructor and Memory Leak** 10 marks

```
#include <iostream>
using namespace std;

class Parent {
public:
~Parent() {
cout << "Parent Destructor" << endl;
}
};

class Child : public Parent {
public:
~Child() {
cout << "Child Destructor" << endl;
}
};

int main() {
Parent *p = new Child();
delete p;
return 0;
}
```

Your Code

```cpp
#include <iostream>
using namespace std;
class Parent {
public:
    virtual ~Parent() {
        cout << "Parent Destructor" << endl;
    }
};
class Child : public Parent {
public:
    ~Child() {
        cout << "Child Destructor" << endl;
    }
};
int main() {
    Parent *p = new Child();
    delete p;
    return 0;
}
```

Input

```
Child
Parent
```

Output

Q4 **Abstract Class and Runtime Polymorphism** 10 marks

```
#include <iostream>
using namespace std;

class Employee {
public:
void calculateSalary() {
cout << "Employee Salary" << endl;
}
};

class FullTimeEmployee : public Employee {
double salary;
public:
FullTimeEmployee(double s) {
salary = s;
}

void calculateSalary() {
cout << salary << endl;
}
};

class PartTimeEmployee : public Employee {
```

Your Code

```
#include <iostream>
using namespace std;
class Employee {
public:
    virtual void calculateSalary() {
        cout << "Employee Salary" << endl;
    }
    virtual ~Employee() {}
};
class FullTimeEmployee : public Employee {
    double salary;

public:
    FullTimeEmployee(double s) {
        salary = s;
    }
void calculateSalary() override {
        cout << salary << endl;
    }
};
class PartTimeEmployee : public Employee {
    int hours;
    double rate;

public:
    PartTimeEmployee(int h, double r) {
```

Input

```
50000
20 500
```

Output

Q5 **Runtime Polymorphism** 10 mar

```
#include <iostream>
using namespace std;

class Shape {
public:
void area() {
cout << "Shape Area" << endl;
}
};

class Rectangle : public Shape {
public:
void area() {
int l, b;
cin >> l >> b;
cout << l * b << endl;
}
};

class Circle : public Shape {
public:
void area() {
int r;
```

Your Code

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual void area() {
        cout << "Shape Area" << endl;
    }
};
class Rectangle : public Shape {
public:
    void area() override {
        int l, b;
        cin >> l >> b;
        cout << l * b << endl;
    }
};
class Circle : public Shape {
public:
    void area() override {
        int r;
        cin >> r;
        cout << 3.14 * r * r << endl;
    }
};
int main() {
    Shape *s;
    Rectangle r;
```

Input

```
5 4
3
```

Output